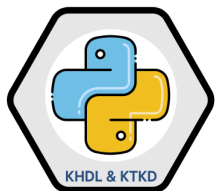


Kiến thức cơ bản về NumPy

Contents

- 5.1. Giới thiệu chung về Numpy
- 5.2. Đối tượng ndarray của NumPy
- 5.3. Hàm xử lý trên từng phần tử trong mảng
- 5.4. Xử lý dữ liệu dựa trên mảng Numpy
- 5.5. Nhập và xuất file với mảng Numpy
- 5.6. Các phép toán đại số tuyến tính



KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

5.1. Giới thiệu chung về Numpy

`NumPy`, là viết tắt của Numerical Python, là một trong những thư viện nền tảng quan trọng nhất cho tính toán số học trong Python. Hầu hết các thư viện tính toán cung cấp các chức năng khoa học, như `SciPy` và `pandas`, đều sử dụng cấu trúc mảng từ `NumPy` làm nền tảng. Mặc dù `pandas` cung cấp các cấu trúc dữ liệu bậc cao phong phú và tiện lợi được xây dựng dựa trên `NumPy`, nhưng trong nhiều trường hợp chúng ta vẫn cần phải làm việc trực tiếp với các đối tượng `NumPy`.

Dưới đây là một số tính năng mà NumPy cung cấp:

- `ndarray`, một đối tượng mảng N-chiều hiệu quả, cung cấp các phép toán **vector hóa** nhanh chóng và khả năng lan truyền linh hoạt.

- Các hàm chuẩn để thực hiện các phép toán trên toàn bộ mảng hoặc trên từng phần tử dữ liệu mà không cần viết vòng lặp.
- Các công cụ để đọc và ghi dữ liệu mảng và làm việc với các tệp được **ánh xạ bộ nhớ** (memory-mapped files).
- Các phép toán đại số tuyến tính, biến đổi Fourier và sinh số ngẫu nhiên.

Ngoài các khả năng tính toán mạnh mẽ mà `NumPy` bổ sung cho Python, một trong những mục tiêu chính của thư viện này khi được tạo ra là để giải quyết vấn đề “hai ngôn ngữ” của Python. Điều này đề cập đến tình huống mà các nhà phát triển cần viết mã `C` hoặc `Fortran` hiệu suất cao để xử lý một lượng lớn dữ liệu, sau đó đưa các câu lệnh đó vào trong Python để dễ sử dụng. `NumPy` được thiết kế để làm cho việc tính toán trên dữ liệu lớn trở nên hiệu quả trong khi vẫn duy trì được tính dễ sử dụng của Python.

Đối với hầu hết các ứng dụng phân tích dữ liệu, mối quan tâm chính đến `NumPy` sẽ tập trung vào các tính năng xử lý véc-tơ, ma trận, mảng nhiều chiều của thư viện này:

- Chuẩn bị dữ liệu: làm sạch, thao tác, chuẩn hóa, định hình lại, sắp xếp và lọc.
- Phân tích và mô hình hóa dữ liệu: áp dụng các phép toán và hàm thống kê và tổng hợp cho toàn bộ tập dữ liệu hoặc các tập con cụ thể. Các thuật toán học máy thường yêu cầu dữ liệu đầu vào phải ở dạng mảng `NumPy`.
- Mô phỏng: tạo dữ liệu ngẫu nhiên theo các phân phối đã biết và chạy các mô hình mô phỏng trên dữ liệu đó.
- Các loại thuật toán số khác: xử lý tín hiệu và lọc.

Mặc dù `NumPy` cung cấp nền tảng tính toán cho nhiều loại ứng dụng khoa học dữ liệu, nhưng bạn đọc sẽ thường sử dụng thư viện `pandas` làm cơ sở cho hầu hết các công việc phân tích dữ liệu, đặc biệt là đối với dữ liệu dạng bảng. `pandas` cũng cung cấp một số chức năng dành riêng cho miền cụ thể hơn, chẳng hạn như xử lý chuỗi thời gian, mà `NumPy` không có. Như vậy, `NumPy` là thành phần quan trọng nhất của hệ sinh thái khoa học dữ liệu Python, nhưng bạn sẽ không nhất thiết phải sử dụng trực tiếp các hàm của `NumPy` mọi lúc.

Trong chương này và xuyên suốt cuốn sách, tôi sẽ sử dụng quy ước chuẩn là nhập `NumPy` dưới tên `np`:

```
import numpy as np
```

Một số bạn đọc có thể đề xuất không thực hiện `import numpy as np` và thay vào đó thực hiện `from numpy import *`. Thực tế thì không gian tên `numpy` rất lớn và chứa nhiều hàm có tên

trùng với các hàm Python tích hợp sẵn (như `min` và `max`). Kết quả có thể dẫn đến các vấn đề khó theo dõi nếu bạn không cẩn thận.

Khi bạn thấy `np.some_function`, điều này có nghĩa là một hàm hoặc đối tượng trong không gian tên cấp cao nhất của `NumPy`. Chúng ta sẽ thảo luận chi tiết về một số thành phần chính của `NumPy`, nhưng bạn đọc có thể khám phá không gian tên bằng cách sử dụng IPython hoặc Jupyter với tính năng tự động hoàn thành bằng phím Tab. Ví dụ, nếu bạn nhập `np.<TAB>`, bạn sẽ thấy một danh sách các mô-đun con (như `np.random`, `np.fft`, `np.linalg`, v.v.) và cả các hàm thường được sử dụng.

5.2. Đối tượng ndarray của NumPy

Một trong những đặc điểm quan trọng của `NumPy` là đối tượng mảng N-chiều, hay `ndarray`. Đây là một đối tượng đa chiều, nhanh chóng và linh hoạt cho các tập dữ liệu lớn trong Python. Các mảng cho phép bạn thực hiện các phép toán toán học trên toàn bộ khối dữ liệu bằng cách sử dụng cú pháp tương tự như các phép toán giữa các phần tử vô hướng. Có rất nhiều điều cần học về các mảng `NumPy`, nhưng chúng ta sẽ bắt đầu với những dạng cơ bản nhất.

Để thấy cách các mảng `NumPy` tạo điều kiện cho các phép toán theo nhóm với cú pháp tối thiểu, trước tiên hãy xem xét một mảng `NumPy` nhỏ. Giả sử chúng ta có một số dữ liệu trong một danh sách Python:

```
my_arr = np.arange(1000000) # Đối tượng mảng của Numpy
my_list = list(range(1000000)) # Đối tượng mảng của Python
```

Bây giờ, giả sử chúng ta muốn nhân mỗi phần tử với 2. Nếu làm điều này với Python thông thường, chúng ta phải lặp qua danh sách đó:

```
for _ in range(10): my_list2 = [x * 2 for x in my_list]
```

Nếu chúng ta thực hiện thao tác tương tự trên một mảng `NumPy`, đoạn câu lệnh sẽ như sau

```
for _ in range(10): my_arr2 = my_arr * 2
```

Vấn đề cốt lõi ở đây là thời gian thực hiện phép tính. Chúng ta có thể đo thời gian giữa việc không sử dụng và có sử dụng `NumPy` như sau:

```
import time
t0 = time.time()
for _ in range(10): my_list2 = [x * 2 for x in my_list]
t1 = time.time()
print(f"List comprehension: {t1 - t0:.4f} s")

t0 = time.time()
for _ in range(10): my_arr2 = my_arr * 2
t1 = time.time()
print(f"NumPy array: {t1 - t0:.4f} s")
```

```
List comprehension: 0.5964 s
NumPy array: 0.0225 s
```

Các phép toán trên véc-tơ của NumPy thường nhanh hơn so với các vòng lặp Python thuần túy trong bất kỳ loại tính toán nào. Trong các phần sau, chúng ta sẽ được tìm hiểu về cơ chế **broadcasting**, hay lan truyền, một tập hợp các quy tắc mạnh mẽ để áp dụng các phép toán giữa các mảng có kích thước khác nhau.

Một `ndarray` là một đối tượng chứa dữ liệu đồng nhất; nghĩa là, tất cả các phần tử của nó phải có cùng một kiểu. Mỗi đối tượng có một phương thức `shape` trả lại giá trị là một `tuple` biểu thị kích thước của mỗi chiều, và phương thức `dtype`, một đối tượng mô tả **kiểu dữ liệu** của mảng:

```
data = np.array([[1.5, -0.1, 3], [0, -3, 6.5]])
data
```

```
array([[ 1.5, -0.1,  3. ],
       [ 0. , -3. ,  6.5]])
```

Sau đó chúng ta có thể viết:

```
data * 10
```

```
array([[ 15., -1.,  30.],
       [ 0., -30.,  65.]])
```

```
data + data
```

```
array([[ 3. , -0.2,  6. ],
       [ 0. , -6. , 13. ]])
```

Và kiểm tra `shape` và `dtype` của nó:

```
data.shape
```

```
(2, 3)
```

```
data.dtype
```

```
dtype('float64')
```

Phần này của chương sẽ giúp bạn làm quen với những điều cơ bản về việc sử dụng mảng `NumPy`.

5.2.1. Tạo một ndarray

Cách dễ nhất để tạo một mảng trong Numpy là sử dụng hàm `np.array`. Chúng ta có thể tạo một mảng như sau:

```
data1 = [6, 7.5, 8, 0, 1]
arr1 = np.array(data1)
arr1
```

```
array([6. , 7.5, 8. , 0. , 1. ])
```

Khi khởi tạo mảng bằng các danh sách lồng nhau, Python sẽ chuyển đổi thành mảng nhiều chiều:

```
data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
arr2 = np.array(data2)
arr2
```

```
array([[1, 2, 3, 4],  
       [5, 6, 7, 8]])
```

Có thể hiểu `data2` là một danh sách của các danh sách, mảng `arr2` có hai chiều với `shape` được suy ra từ dữ liệu. Chúng ta có thể xác nhận điều này bằng cách kiểm tra các thuộc tính `ndim` và `shape`:

```
arr2.ndim
```

```
2
```

```
arr2.shape
```

```
(2, 4)
```

Trừ khi được chỉ định rõ ràng, hàm `np.array` cố gắng suy ra kiểu dữ liệu phù hợp cho mảng mà nó tạo ra. Kiểu dữ liệu được lưu trữ trong một đối tượng `dtype`. Trong hai ví dụ trước chúng ta có:

```
arr1.dtype
```

```
dtype('float64')
```

```
arr2.dtype
```

```
dtype('int64')
```

Ngoài `np.array`, có một số hàm khác để tạo mảng mới.

- `np.zeros` và `np.ones` tạo ra các mảng chỉ có các số 0 hoặc số 1 tương ứng với một chiều dài hoặc `shape` cho trước.
- `np.empty` tạo ra một mảng mà không khởi tạo các giá trị của nó cho bất kỳ giá trị cụ thể nào. Để tạo một mảng có `shape` cao hơn, hãy gán một `tuple` cho `shape`:

```
np.zeros(10)
```

```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
np.zeros((3, 6))
```

```
array([[0., 0., 0., 0., 0., 0.],  
       [0., 0., 0., 0., 0., 0.],  
       [0., 0., 0., 0., 0., 0.]])
```

```
np.empty((2, 3, 2))
```

```
array([[[1.05624525e-311, 3.16202013e-322],  
        [0.00000000e+000, 0.00000000e+000],  
        [1.11260619e-306, 1.65780572e-076]],  
       [[3.37673009e-057, 8.00845464e+165],  
        [3.85726607e-057, 9.06006654e-043],  
        [6.87749148e+169, 2.67784158e+184]]])
```

Chú ý: Mặc dù `np.empty` trả về một mảng giá trị, nhưng hoàn toàn không có ý nghĩa. Tiếp cận tốt nhất là gán giá trị cho mảng sau khi khởi tạo.

`np.arange` là một phiên bản giống như hàm `range` tích hợp sẵn của Python, nhưng trả về một mảng `Numpy` thay vì một danh sách của Python:

```
np.arange(15)
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

Xem bảng ... để biết danh sách một phần các hàm tạo mảng chuẩn.

bảng ...: một số hàm tạo mảng NumPy quan trọng

Hàm	Mô tả
<code>array</code>	Chuyển đổi dữ liệu đầu vào (danh sách, tuple, mảng hoặc chuỗi khác) thành một ndarray, suy ra một dtype cho ndarray hoặc chỉ định rõ ràng một dtype. Dữ liệu được sao chép theo mặc định.
<code>asarray</code>	Chuyển đổi đầu vào thành ndarray, nhưng không sao chép nếu đầu vào đã là một ndarray có dtype phù hợp.
<code>arange</code>	Giống như <code>range</code> tích hợp sẵn nhưng trả về một ndarray thay vì một danh sách.
<code>ones</code> , <code>ones_like</code>	Tạo một mảng chứa toàn số 1 với <code>shape</code> và <code>dtype</code> cho trước. <code>ones_like</code> lấy một mảng khác và tạo ra một mảng chứa toàn số 1 có cùng <code>shape</code> và <code>dtype</code> .
<code>zeros</code> , <code>zeros_like</code>	Giống như <code>ones</code> và <code>ones_like</code> nhưng tạo ra các mảng chứa toàn số 0.
<code>empty</code> , <code>empty_like</code>	Tạo các mảng mới bằng cách cấp phát bộ nhớ mới, nhưng không điền bất kỳ giá trị nào như <code>ones</code> và <code>zeros</code> .
<code>full</code> , <code>full_like</code>	Tạo một mảng có <code>shape</code> và <code>dtype</code> cho trước với tất cả các giá trị được đặt thành "giá trị điền" được chỉ định. <code>full_like</code> lấy một mảng khác và tạo ra một mảng được điền có cùng <code>shape</code> và <code>dtype</code> .
<code>eye</code> , <code>identity</code>	Tạo một ma trận đơn vị (identity matrix) vuông $N \times N$ (1 trên đường chéo và 0 ở những nơi khác).

5.2.2. Kiểu dữ liệu của ndarray

Kiểu dữ liệu, hay `dtype`, là một đối tượng đặc biệt chứa thông tin mà `ndarray` cần để diễn giải một khối bộ nhớ là một loại dữ liệu cụ thể:

```
arr1_dtype = np.array([1, 2, 3], dtype=np.float64)
arr2_dtype = np.array([1, 2, 3], dtype=np.int32)
arr1_dtype.dtype
```



```
dtype('float64')
```

```
arr2_dtype.dtype
```

```
dtype('int32')
```

Các `dtype` là lý do tại sao `NumPy` rất mạnh mẽ và linh hoạt. Trong hầu hết các trường hợp, `dtype` giúp dễ dàng đọc và ghi dữ liệu dưới dạng nhị phân vào máy và cũng dễ dàng kết nối với các câu lệnh được viết bằng các ngôn ngữ như `C` hoặc `Fortran`.

Các `dtype` kiểu số được đặt tên giống nhau là `float` hoặc `int`, theo sau là một số cho biết số bit trên mỗi phần tử. Bạn đọc xem Bảng dưới đây để biết danh sách đầy đủ các `dtype` được hỗ trợ.

Bảng ...: Các kiểu dữ liệu NumPy

Kiểu	Mã kiểu	Mô tả
<code>int8</code> , <code>uint8</code>	<code>i1</code> , <code>u1</code>	Số nguyên 8 bit có dấu và không dấu
<code>int16</code> , <code>uint16</code>	<code>i2</code> , <code>u2</code>	Số nguyên 16 bit có dấu và không dấu
<code>int32</code> , <code>uint32</code>	<code>i4</code> , <code>u4</code>	Số nguyên 32 bit có dấu và không dấu
<code>int64</code> , <code>uint64</code>	<code>i8</code> , <code>u8</code>	Số nguyên 64 bit có dấu và không dấu
<code>float16</code>	<code>f2</code>	Số thực dấu phẩy động nửa độ chính xác (half-precision)
<code>float32</code>	<code>f4</code> hoặc <code>f</code>	Số thực dấu phẩy động đơn độ chính xác (single-precision) tiêu chuẩn; tương thích với <code>float</code> trong C
<code>float64</code>	<code>f8</code> hoặc <code>d</code>	Số thực dấu phẩy động độ chính xác kép (double-precision) tiêu chuẩn; tương thích với <code>double</code> trong C và <code>float</code> trong Python
<code>float128</code>	<code>f16</code> hoặc <code>g</code>	Số thực dấu phẩy động độ chính xác mở rộng (extended-precision)
<code>complex64</code> , <code>complex128</code> , <code>complex256</code>	<code>c8</code> , <code>c16</code> , <code>c32</code>	Số phức được biểu diễn bằng hai số thực dấu phẩy động 32, 64, hoặc 128 bit tương ứng
<code>bool</code>	<code>?</code>	Kiểu Boolean lưu trữ các giá trị <code>True</code> và <code>False</code>
<code>object</code>	<code>0</code>	Kiểu đối tượng Python; một giá trị có thể là bất kỳ đối tượng Python nào
<code>string_</code>	<code>S</code>	Kiểu chuỗi ASCII có độ dài cố định (ví dụ, để khớp với <code>dtype</code> khi đọc một tệp). Độ dài của chuỗi được chỉ định bởi số theo

Kiểu	Mã kiểu	Mô tả
		sau, ví dụ, <code>S10</code> là một chuỗi 10 ký tự. NumPy không cung cấp <code>dtype</code> chuỗi Unicode có độ dài cố định.
<code>unicode_</code>	<code>U</code>	Kiểu Unicode có độ dài cố định (ví dụ, <code>U10</code>). NumPy không cung cấp <code>dtype</code> chuỗi Unicode có độ dài thay đổi, không giống như pandas.

Bạn có thể chuyển đổi, hay còn gọi là `cast`, một mảng từ `dtype` này sang `dtype` khác bằng cách sử dụng phương thức `astype` của `ndarray`:

```
arr_astype = np.array([1, 2, 3, 4, 5])
arr_astype.dtype
```

```
dtype('int64')
```

```
float_arr = arr_astype.astype(np.float64)
float_arr.dtype
```

```
dtype('float64')
```

Trong ví dụ này, các số nguyên được đổi kiểu thành số thực dấu phẩy động. Ngược lại, nếu chúng ta ép kiểu một số số thực dấu phẩy động thành kiểu số nguyên, phần thập phân sẽ bị cắt bỏ:

```
arr_fp_to_int = np.array([3.7, -1.2, -2.6, 0.5, 12.9, 10.1])
arr_fp_to_int
```

```
array([ 3.7, -1.2, -2.6,  0.5, 12.9, 10.1])
```

```
arr_fp_to_int.astype(np.int32)
```

```
array([ 3, -1, -2,  0, 12, 10], dtype=int32)
```

Nếu bạn có một mảng kiểu chuỗi ký tự nhưng bản chất là biểu diễn các số, bạn có thể sử dụng `astype` để chuyển đổi chúng thành dạng số:

```
numeric_strings = np.array(["1.25", "-9.6", "42"], dtype=np.string_)
numeric_strings.astype(float) # NumPy sẽ tự động suy ra np.float64
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 numeric_strings = np.array(["1.25", "-9.6", "42"], dtype=np.string_)
      2 numeric_strings.astype(float) # NumPy sẽ tự động suy ra np.float64

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\__init__.py:413:
   410     raise AttributeError(__former_attrs__[attr], name=None)
   412 if attr in __expired_attributes__:
--> 413     raise AttributeError(
   414         f"`np.{attr}` was removed in the NumPy 2.0 release. "
   415         f"{__expired_attributes__[attr]}",
   416         name=None
   417     )
   419 if attr == "chararray":
   420     warnings.warn(
   421         "`np.chararray` is deprecated and will be removed from "
   422         "the main namespace in the future. Use an array with a string "
   423         "or bytes dtype instead.", DeprecationWarning, stacklevel=2)

AttributeError: `np.string_` was removed in the NumPy 2.0 release. Use `np.bytes_` inst
```

Lưu ý: Việc gọi `astype` luôn tạo ra một mảng mới, hay một bản sao của dữ liệu, ngay cả khi `dtype` mới giống với `dtype` cũ.

Bạn đọc cần lưu ý khi sử dụng `dtype = np.string_`, vì dữ liệu chuỗi trong `Numpy` có độ dài cố định và có thể bị cắt bớt mà không có cảnh báo.

Bạn cũng có thể sử dụng mã kiểu `dtype` thay cho tên đầy đủ:

```
empty_uint32 = np.empty(8, dtype="u4")
empty_uint32
```

```
array([          0,          0, 3236760048,          511, 3236761808,
        511, 3236762128,          511], dtype=uint32)
```

Lưu ý: Nếu bạn gặp khó khăn trong việc tìm ra `dtype` nào nên sử dụng với dữ liệu của mình thì không cần phải quá quan tâm. Bạn có thể dùng `float64` cho hầu hết dữ liệu số thực dấu phẩy động. Việc sử dụng các `dtype` như `float32` thường là để tiết kiệm bộ nhớ khi làm việc với các tập dữ liệu rất lớn.

5.2.3. Các phép toán số học với mảng `NumPy`

Đối tượng mảng rất quan trọng vì chúng cho phép chúng ta thể hiện các phép toán theo nhóm trên dữ liệu mà không cần viết bất kỳ vòng lặp `for` nào. `NumPy` gọi tính năng này là **véc-tơ hóa**, hay vectorization. Bất kỳ phép toán số học nào giữa các mảng có kích thước bằng nhau đều áp dụng phép toán đó theo từng phần tử:

```
arr_arith = np.array([[1., 2., 3.], [4., 5., 6.]])
arr_arith
```

```
array([[1., 2., 3.],
       [4., 5., 6.]])
```

```
arr_arith * arr_arith
```

```
array([[ 1.,  4.,  9.],
       [16., 25., 36.]])
```

```
arr_arith - arr_arith
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

Các phép toán số học với các đại lượng vô hướng truyền giá trị vô hướng đó cho mỗi phần tử trong mảng khi thực hiện phép toán:

```
1 / arr_arith
```

```
arr_arith ** 2
```

Các phép so sánh giữa các mảng có cùng kích thước tạo ra một mảng kiểu logical

```
arr2_arith = np.array([[0., 4., 1.], [7., 2., 12.]])  
arr2_arith
```

```
arr2_arith > arr_arith
```

Việc thực hiện các phép toán giữa các mảng có kích thước khác nhau được gọi là *broadcasting* và sẽ được thảo luận chi tiết hơn trong phần sau

5.2.4. Chỉ số trong mảng

Chỉ số của mảng `NumPy` là một chủ đề rộng, vì có nhiều cách bạn có thể chọn một tập con dữ liệu của mình hoặc các phần tử riêng lẻ. Các mảng một chiều khá đơn giản, chúng hoạt động tương tự như một `list` trong Python:

```
arr_slice_basic = np.arange(10)  
arr_slice_basic
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
arr_slice_basic[5]
```

```
arr_slice_basic[5:8]
```

```
arr_slice_basic[5:8] = 12  
arr_slice_basic
```

```
array([ 0,  1,  2,  3,  4, 12, 12, 12,  8,  9])
```

Như bạn đọc thấy, nếu bạn gán một giá trị vô hướng cho một lát cắt, như trong `arr[5:8] = 12`, giá trị đó được truyền, hay còn gọi là *broadcasted* cho toàn bộ giá trị trong lát cắt. Sự khác biệt quan trọng đầu tiên so với `list` tích hợp sẵn trong Python là các lát cắt mảng là các **khung nhìn** trên mảng gốc. Điều này có nghĩa là dữ liệu không được sao chép, và bất kỳ sửa đổi nào đối với khung nhìn sẽ được phản ánh trong mảng ban đầu. Để cung cấp cho bạn một ví dụ về điều này, trước tiên chúng ta tạo một lát cắt của `arr_slice_basic`:

```
arr_slice_view = arr_slice_basic[5:8]  
arr_slice_view
```

```
array([12, 12, 12])
```

Bây giờ, khi chúng ta thay đổi các giá trị trong `arr_slice_view`, các thay đổi đó được phản ánh trong mảng gốc `arr_slice_basic`:

```
arr_slice_view[1] = 12345  
arr_slice_basic # Giá trị tương ứng với chỉ số 6 sẽ thay đổi
```

```
array([ 0,  1,  2,  3,  4, 12, 12345, 12,  8,  
       9])
```

Sử dụng chỉ số kiểu `[:]` sẽ gán cho tất cả các giá trị trong một mảng:

```
arr_slice_view[:] = 64  
arr_slice_basic
```

Nếu bạn muốn một bản sao của một lát cắt của một `ndarray` thay vì một khung nhìn, bạn sẽ cần phải sao chép mảng một cách rõ ràng, chẳng hạn như sử dụng

```
arr_slice_basic[5:8].copy().
```

Với các mảng có chiều cao hơn, bạn có nhiều tùy chọn hơn. Trong một mảng hai chiều, các phần tử tại mỗi chỉ mục không phải là các đại lượng vô hướng mà là các mảng một chiều:

```
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
arr2d[2]
```

```
array([7, 8, 9])
```

Do đó, các phần tử riêng lẻ có thể được truy cập bằng cách sử dụng các chỉ số theo kiểu đệ quy. Hoặc bạn đọc có thể truyền một danh sách các chỉ số được phân tách bằng dấu phẩy để chọn một phần tử riêng lẻ. Vì vậy, những cách truy cập sau đây là tương đương:

```
arr2d[0][2]
```

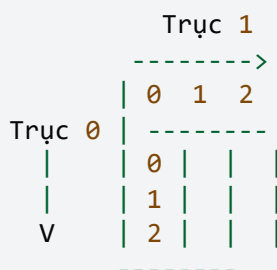
```
np.int64(3)
```

```
arr2d[0, 2]
```

```
np.int64(3)
```

Hình ... minh họa về việc truy cập các phần tử một mảng hai chiều theo chỉ số. Bạn đọc có thể nghĩ trục 0 tương đương với "hàng" và trục 1 tương đương với "cột".

Hình ... Chỉ số các phần tử trong một mảng NumPy hai chiều



Một cách tổng quát, trong các mảng nhiều chiều, nếu bạn bỏ qua các chỉ số phía sau, đối tượng được trả về sẽ là một `ndarray` có chiều thấp hơn bao gồm tất cả dữ liệu dọc theo các chiều cao hơn. Ví dụ, hãy quan sát mảng $2 \times 2 \times 3$ `arr3d` như sau:


```
arr3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
arr3d
```

```
array([[[ 1,  2,  3],
         [ 4,  5,  6]],
       [[ 7,  8,  9],
         [10, 11, 12]]])
```

`arr3d[0]` sẽ là một mảng 2×3 :

```
arr3d[0]
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

Cả các giá trị số và các mảng đều có thể được gán cho `arr3d[0]`:

```
old_values = arr3d[0].copy() # lưu lại giá trị cũ
arr3d[0] = 42 # gán mảng bằng một số
arr3d
```

```
arr3d[0] = old_values # Trả lại giá trị ban đầu
arr3d
```

Tương tự, `arr3d[1, 0]` cung cấp tất cả các giá trị có chỉ mục bắt đầu bằng `(1, 0)`, tạo thành một mảng một chiều có 3 phần tử:

```
arr3d[1, 0]
```

```
array([7, 8, 9])
```

Biểu thức này giống như khi chúng ta tạo một khung hình `arr3d[1]` và sau đó tiếp tục tạo một khung hình khác với các chỉ số thích hợp:

```
x = arr3d[1]
x
```

```
array([[ 7,  8,  9],
       [10, 11, 12]])
```

```
x[0]
```

```
array([7, 8, 9])
```

Lưu ý rằng trong tất cả các trường hợp các mảng được trả về là đều các khung nhìn.

5.2.4.1. Sử dụng lát cắt trên mảng

Giống như các đối tượng một chiều, chẳng hạn như danh sách trong Python, `ndarray` có thể được cắt lát bằng cú pháp quen thuộc:

```
arr_slice_basic # arr_slice_basic đã được định nghĩa ở trên: np.arange(10) và đã bị th
```

```
arr_slice_basic[1:6]
```

Hãy xem xét `arr2d` từ trước. Bạn có thể truyền nhiều lát cắt giống như bạn có thể truyền nhiều chỉ mục:

```
arr2d # arr2d đã được định nghĩa ở trên
```

```
arr2d[:2]
```

Bạn có thể truyền nhiều lát cắt theo nhiều trục. Ví dụ:

```
arr2d[:2, 1:]
```

Khi cắt lát như thế này, bạn đọc luôn nhận được các khung hình là mảng có cùng số chiều. Bằng cách sử dụng đồng thời các chỉ mục số nguyên và lát cắt, bạn có thể nhận được các lát cắt có chiều thấp hơn.

Ví dụ, chúng ta có thể chọn hàng thứ hai nhưng chỉ hai cột đầu tiên:

```
arr2d[1, :2]
```

```
array([4, 5])
```

Tương tự, cũng có thể chọn cột thứ ba nhưng chỉ hai hàng đầu tiên:

```
arr2d[:2, 2]
```

```
array([3, 6])
```

Xem Hình ... để biết minh họa. Lưu ý rằng dấu hai chấm (:) đứng một mình có nghĩa là lấy toàn bộ trục, vì vậy bạn có thể cắt lát chỉ các chiều cao hơn như sau:

```
arr2d[:, :1]
```

Hình ...: Các lựa chọn mảng hai chiều từ các lát cắt

```
arr2d = np.array([, ,])  
  
arr2d[:2, 1:] -> [, ]  
arr2d      ->  
arr2d[2, :] ->  
arr2d[2, 1:] ->  
arr2d[:, :2] -> [, ,]  
arr2d[1, :2] ->  
arr2d[1:2, :2] -> [ ]
```

(Hình ảnh trực quan hóa các lát cắt khác nhau trên một mảng 3x3)

Tất nhiên, việc gán cho một biểu thức lát cắt sẽ gán cho toàn bộ lựa chọn:

```
arr2d_copy = arr2d.copy() # Tạo bản sao để không ảnh hưởng arr2d gốc cho các ví dụ sau
arr2d_copy[:, 1:] = 0
arr2d_copy
```

```
array([[1, 0, 0],
       [4, 0, 0],
       [7, 8, 9]])
```

5.2.5. Sử dụng chỉ số kiểu logical

Hãy xem xét một ví dụ trong đó chúng ta có một dữ liệu được lưu trong một mảng và một mảng khác chứa tên các phần tử của mảng dữ liệu:

```
names = np.array(["Bob", "Joe", "Will", "Bob", "Will", "Joe", "Joe"])
data_bool = np.array([[4, 7], [0, 2], [-5, 6], [0, 0], [1, 2], [-12, -4], [3, 4]])
names
data_bool
```

Giả sử mỗi tên trong mảng `names` tương ứng với một hàng trong mảng `data_bool` và chúng ta muốn chọn tất cả các hàng có tên tương ứng là `'Bob'`. Giống như các phép toán số học, các phép so sánh với các mảng, chẳng hạn như `==`, cũng được véc-tơ hóa. Do đó, việc so sánh `names` với chuỗi `'Bob'` tạo ra một mảng kiểu logical:

```
names == "Bob"
```

Mảng logical này có thể được sử dụng như chỉ số khi muốn lấy ra một mảng con:

```
data_bool[names == "Bob"]
```

Mảng logical dùng làm chỉ số phải có cùng độ dài với trực sử dụng làm chỉ số. Chúng ta thậm chí có thể kết hợp các mảng boolean và các lát cắt hoặc chỉ số kiểu số nguyên để chọn mảng con.

Chú ý: Sự khác nhau cơ bản giữa chỉ số bằng logical với cắt lát là chỉ số kiểu logical *luôn* tạo ra một bản sao của dữ liệu, ngay cả khi mảng được trả về không thay đổi.

Mẹo: Từ khóa `and` và `or` của Python không hoạt động với các mảng kiểu logical. Thay vào đó, hãy sử dụng `&` (và) và `|` (hoặc).

Ví dụ, giả sử chúng ta muốn chọn các hàng mà `names == "Bob"` đồng thời lấy chỉ số cột theo kiểu cắt lát:

```
data_bool[names == "Bob", 1:]
```

```
data_bool[names == "Bob", 0]
```

Để chọn mọi tên ngoại trừ `'Bob'`, bạn đọc có thể sử dụng `!=` hoặc phủ định điều kiện bằng `~`:

```
names != "Bob"
```

```
~(names == "Bob") # cách viết khác
```

```
data_bool[~(names == "Bob")]
```

Toán tử `~` sẽ hữu ích khi bạn đọc muốn đảo ngược một điều kiện chung:

```
cond = names == "Bob"  
data_bool[~cond]
```

Để kết hợp các điều kiện logical, bạn đọc sử dụng toán tử số học `&` và `|`. Ví dụ, khi chúng ta muốn tên hàng là `Bob` hoặc `Will`:

```
mask = (names == "Bob") | (names == "Will")  
mask
```

```
data_bool[mask]
```

Việc thay thế giá trị trong mảng sử dụng chỉ số kiểu mảng logical hoạt động theo cách thông thường. Ví dụ, để đặt tất cả các giá trị âm trong `data_bool` thành 0, chúng ta thực hiện như sau:

```
data_bool_copy = data_bool.copy() # Tạo bản sao để không ảnh hưởng data_bool gốc
data_bool_copy[data_bool_copy < 0] = 0
data_bool_copy
```

5.2.6. Chỉ số mảng nâng cao

Kỹ thuật chỉ số nâng cao, hay fancy indexing, là một thuật ngữ được NumPy sử dụng để mô tả việc tạo chỉ số bằng các mảng số nguyên. Giả sử chúng ta có một mảng 8×4 :

```
arr_fancy = np.zeros((8, 4))
for i in range(8):
    arr_fancy[i] = i
arr_fancy
```

```
array([[0., 0., 0., 0.],
       [1., 1., 1., 1.],
       [2., 2., 2., 2.],
       [3., 3., 3., 3.],
       [4., 4., 4., 4.],
       [5., 5., 5., 5.],
       [6., 6., 6., 6.],
       [7., 7., 7., 7.]])
```

Để chọn một tập con các hàng theo một thứ tự cụ thể, bạn đọc cần tạo một danh sách, hoặc một `ndarray`, bao gồm các chỉ số là số nguyên chỉ định thứ tự mong muốn:

```
arr_fancy[[4, 3, 0, 6]]
```

```
array([[4., 4., 4., 4.],
       [3., 3., 3., 3.],
       [0., 0., 0., 0.],
       [6., 6., 6., 6.]])
```

Khi sử dụng các chỉ số âm, các hàng sẽ được lựa chọn theo thứ tự từ hàng cuối:

```
arr_fancy[[-3, -5, -7]]
```

Khi chúng ta sử dụng chỉ số là mảng nhiều chiều kết quả trả lại sẽ là một mảng một chiều các phần tử tương ứng với mỗi chỉ số. Hãy quan sát ví dụ:

```
arr_fancy_reshape = np.arange(32).reshape((8, 4))  
arr_fancy_reshape
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11],  
       [12, 13, 14, 15],  
       [16, 17, 18, 19],  
       [20, 21, 22, 23],  
       [24, 25, 26, 27],  
       [28, 29, 30, 31]])
```

```
arr_fancy_reshape[[1, 5, 7, 2], [0, 3, 1, 2]]
```

```
array([ 4, 23, 29, 10])
```

Ở đây các phần tử có chỉ số tương ứng với `(1, 0)`, `(5, 3)`, `(7, 1)`, và `(2, 2)` đã được chọn. Một lưu ý khác là kết quả của chỉ số nâng cao luôn là một bản sao một chiều của dữ liệu, bất kể số chiều của mảng.

Đây là một ví dụ khác về cách chọn một tập hợp con hình chữ nhật của các hàng và cột:

```
arr_fancy_reshape[[1, 5, 7, 2]][:, [0, 3, 1, 2]]
```

```
array([[ 4,  7,  5,  6],  
       [20, 23, 21, 22],  
       [28, 31, 29, 30],  
       [ 8, 11,  9, 10]])
```

Hãy nhớ rằng khi sử dụng chỉ số kiểu mảng nhiều chiều dữ liệu sẽ luôn được sao chép vào một mảng mới. Nếu bạn cần một khung hình, bạn sẽ cần sử dụng các phương pháp khác, chẳng hạn như một tập hợp các lát cắt.

Việc gán giá trị của mảng sử dụng chỉ số nâng cao hoạt động như thông thường:

```
arr_fancy_reshape_copy = arr_fancy_reshape.copy()
arr_fancy_reshape_copy[[1, 5, 7, 2], [0, 3, 1, 2]] = 0
arr_fancy_reshape_copy
```

```
array([[ 0,  1,  2,  3],
       [ 0,  5,  6,  7],
       [ 8,  9,  0, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22,  0],
       [24, 25, 26, 27],
       [28,  0, 30, 31]])
```

Giả sử chúng ta muốn chọn các hàng theo một thứ tự cụ thể, và sau đó sắp xếp lại các cột. Một cách để làm điều này là:

```
arr_fancy_ix = np.arange(32).reshape((8, 4))
arr_fancy_ix[[1, 5, 7, 2]] # Chọn các hàng
```

```
array([[ 4,  5,  6,  7],
       [20, 21, 22, 23],
       [28, 29, 30, 31],
       [ 8,  9, 10, 11]])
```

```
arr_fancy_ix[[1, 5, 7, 2]][:, [0, 3, 1, 2]] # Chọn các hàng và sau đó sắp xếp lại các
```

```
array([[ 4,  7,  5,  6],
       [20, 23, 21, 22],
       [28, 31, 29, 30],
       [ 8, 11,  9, 10]])
```

Để làm điều này theo một cách ngắn gọn hơn, bạn có thể sử dụng `np.ix_`. Hàm này chuyển đổi hai mảng số nguyên một chiều thành một kiểu chỉ số được sử dụng để lựa chọn một vùng hình chữ nhật từ một mảng:

```
arr_fancy_ix[np.ix_([1, 5, 7, 2], [0, 3, 1, 2])]
```



```
array([[ 4,  7,  5,  6],
       [20, 23, 21, 22],
       [28, 31, 29, 30]])
```

Bạn đọc có thể thấy rằng hàm `np.ix` tạo ra một đối tượng chỉ số để lựa chọn các hàng 1, 5, 7, 2 và các cột 0, 3, 1, 2. Điều này tương đương với việc chọn các hàng với `arr[[1, 5, 7, 2], :]` và sau đó chọn các cột với `[:, [0, 3, 1, 2]]` trên kết quả.

5.2.7. Chuyển vị mảng và hoán đổi thứ tự trục

Chuyển vị là một dạng đặc biệt của việc định hình lại, hay reshaping, đồng thời trả về một khung hình của dữ liệu ban đầu. Các mảng có thuộc tính `T` và phương thức `transpose`:

```
arr_T = np.arange(15).reshape((3, 5))
arr_T
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

```
arr_T.T
```

```
array([[ 0,  5, 10],
       [ 1,  6, 11],
       [ 2,  7, 12],
       [ 3,  8, 13],
       [ 4,  9, 14]])
```

Khi thực hiện các phép toán đại số với ma trận, bạn đọc thường xuyên tính toán inner product, hay phép nhân ma trận giữa ma trận ban đầu với chuyển vị của nó. Phép nhân ma trận trong Numpy được thực hiện bằng hàm `np.dot`:

```
arr_dot_T = np.array([[0, 1, 0], [1, 2, -2], [6, 3, 2], [-1, 0, -1], [1, 0, 1]])
arr_dot_T
```

```
array([[ 0,  1,  0],
       [ 1,  2, -2],
       [ 6,  3,  2],
       [-1,  0, -1],
       [ 1,  0,  1]])
```

```
np.dot(arr_dot_T.T, arr_dot_T)
```

```
array([[39, 20, 12],
       [20, 14,  2],
       [12,  2, 10]])
```

Phương thức `transpose` được dùng một cách tổng quát hơn trên mảng nhiều chiều để hoán vị các trục trong khi thuộc tính `T` chỉ được sử dụng với các mảng hai chiều. Hãy quan sát mảng ba chiều sau:

```
arr_transpose_axes = np.arange(16).reshape((2, 2, 4))
arr_transpose_axes
```

```
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7]],
       [[ 8,  9, 10, 11],
        [12, 13, 14, 15]])
```

```
arr_transpose_axes.transpose((1, 0, 2)) # Hoán đổi trục 0 và trục 1
```

Ở đây, các trục đã được sắp xếp lại với trục thứ hai là trục đầu tiên, trục đầu tiên là trục thứ hai và trục cuối cùng không thay đổi.

Chỉ số đơn giản bằng `arr[i]` sẽ chọn dữ liệu dọc theo trục 0. Để chọn dữ liệu dọc theo các trục khác, bạn có thể cần phải thực hiện một số định hình lại hoặc chuyển vị. Các mảng `ndarray` có một phương thức `swapaxes` nhận một cặp số trục và chuyển đổi các trục được chỉ định để sắp xếp lại dữ liệu:

```
arr_transpose_axes # arr_transpose_axes từ ô trên
```

```
arr_transpose_axes.swapaxes(0, 1)
```

`swapaxes` tương tự cũng trả về một khung hình trên dữ liệu mà không tạo bản sao.

5.3. Hàm xử lý trên từng phần tử trong mảng

Các hàm được gọi là **universal function**, hay viết tắt là *ufunc*, là các hàm thực hiện các phép toán theo từng phần tử trên dữ liệu trong `ndarray`. *ufunc* có đặc điểm là được tối ưu hóa để xử lý phần tử theo từng phần tử với hiệu suất cao và hỗ trợ broadcasting, vector hóa, và multidimensional arrays.

Nhiều ufunc rất đơn giản, thực hiện theo từng phần tử, như `np.sqrt` hoặc `np.exp`:

```
arr_ufunc = np.arange(10)
arr_ufunc
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.sqrt(arr_ufunc)
```

```
array([0.          , 1.          , 1.41421356, 1.73205081, 2.          ,
       2.23606798, 2.44948974, 2.64575131, 2.82842712, 3.          ])
```

```
np.exp(arr_ufunc)
```

```
array([1.00000000e+00, 2.71828183e+00, 7.38905610e+00, 2.00855369e+01,
       5.45981500e+01, 1.48413159e+02, 4.03428793e+02, 1.09663316e+03,
       2.98095799e+03, 8.10308393e+03])
```

Những hàm như `np.sqrt` hay `np.exp` được gọi là các *unary ufuncs*, nghĩa là chỉ thực hiện tính toán trên từng phần tử của một mảng. Những hàm khác, chẳng hạn như `np.add` hoặc `np.maximum`, nhận hai mảng và trả về một mảng duy nhất làm kết quả. Các hàm này được gọi là *binary ufunc*

```
x_ufunc = np.random.standard_normal(8)
y_ufunc = np.random.standard_normal(8)
x_ufunc
```

```
y_ufunc
```

```
np.maximum(x_ufunc, y_ufunc)
```

Ở trên, `np.maximum` đã tính toán giá trị lớn nhất theo từng phần tử trong `x_ufunc` và `y_ufunc`.

Trong khi không phổ biến, một ufunc có thể trả về nhiều mảng. `np.modf` là một ví dụ: một ufunc đơn phân trả về phần thập phân và phần nguyên của một mảng chứa số thực dấu phẩy động:

```
arr_modf = np.random.standard_normal(7) * 5
arr_modf
```

```
array([ 7.94970435, -2.12568793, -3.47077194, -2.90469311,
        0.21465915, -11.93256917, -3.02877849])
```

```
remainder, whole_part = np.modf(arr_modf)
remainder
```

```
array([ 0.94970435, -0.12568793, -0.47077194, -0.90469311,  0.21465915,
       -0.93256917, -0.02877849])
```

```
whole_part
```

```
array([ 7., -2., -3., -2.,  0., -11., -3.])
```

Ufunc chấp nhận một đối số `out` tùy chọn cho phép chúng hoạt động tại chỗ trên các mảng:

```
arr_modf # arr_modf từ ô trước
```

```
array([ 7.94970435, -2.12568793, -3.47077194, -2.90469311,  
       0.21465915, -11.93256917, -3.02877849])
```

```
out_arr = np.zeros_like(arr_modf)  
np.add(arr_modf, 1, out=out_arr) # out_arr += arr + 1, không tạo mảng mới  
out_arr
```

```
array([ 8.94970435, -1.12568793, -2.47077194, -1.90469311,  
       1.21465915, -10.93256917, -2.02877849])
```

Các bảng ... và ... liệt kê các unary ufunc và binary ufunc thường dùng trong NumPy. Một số trong số này sẽ được sử dụng trong các ví dụ trong suốt cuốn sách.

Bảng ... một số unary ufunc

Hàm	Mô tả
<code>abs</code> , <code>fabs</code>	Tính giá trị tuyệt đối theo từng phần tử cho số nguyên, số thực dấu phẩy động hoặc số phức. Sử dụng <code>fabs</code> cho dữ liệu số thực dấu phẩy động không phức (trả về một mảng số thực dấu phẩy động).
<code>sqrt</code>	Tính căn bậc hai theo từng phần tử. Tương đương với <code>arr ** 0.5</code> .
<code>square</code>	Tính bình phương theo từng phần tử. Tương đương với <code>arr ** 2</code> .
<code>exp</code>	Tính lũy thừa e^x theo từng phần tử.
<code>log</code> , <code>log10</code> , <code>log2</code> , <code>log1p</code>	Logarit tự nhiên (cơ số e), logarit cơ số 10, logarit cơ số 2, và $\log(1 + x)$ tương ứng.
<code>sign</code>	Tính dấu của mỗi phần tử: 1 (dương), 0 (zero), hoặc -1 (âm).
<code>ceil</code>	Tính trần theo từng phần tử (giá trị nguyên nhỏ nhất lớn hơn hoặc bằng mỗi phần tử).
<code>floor</code>	Tính sàn theo từng phần tử (giá trị nguyên lớn nhất nhỏ hơn hoặc bằng mỗi phần tử).
<code>rint</code>	Làm tròn các phần tử đến số nguyên gần nhất, giữ nguyên <code>dtype</code> .
<code>modf</code>	Trả về phần phân số và phần nguyên của mảng dưới dạng các mảng riêng biệt.
<code>isnan</code>	Trả về một mảng boolean cho biết mỗi giá trị có phải là <code>NaN</code> (Not a Number) hay không.
<code>isfinite</code> , <code>isinf</code>	Trả về các mảng boolean cho biết mỗi phần tử có hữu hạn (không- <code>inf</code> , không- <code>NaN</code>) hoặc vô hạn tương ứng.
<code>cos</code> , <code>cosh</code> , <code>sin</code> , <code>sinh</code> , <code>tan</code> , <code>tanh</code>	Các hàm lượng giác thông thường và các hàm hypebolic tương ứng.

Hàm	Mô tả
<code>arccos</code> , <code>arccosh</code> , <code>arcsin</code> , <code>arcsinh</code> , <code>arctan</code> , <code>arctanh</code>	Các hàm lượng giác nghịch đảo.
<code>logical_not</code>	Tính giá trị chân lý của <code>not x</code> theo từng phần tử. Tương đương với <code>~arr</code> .

Bảng ... một số binary ufunc

Hàm	Mô tả
<code>add</code>	Cộng các phần tử tương ứng trong các mảng.
<code>subtract</code>	Trừ các phần tử trong mảng thứ hai khỏi mảng thứ nhất, theo từng phần tử.
<code>multiply</code>	Nhân các phần tử của mảng, theo từng phần tử.
<code>divide</code> , <code>floor_divide</code>	Chia hoặc chia lấy phần nguyên (làm tròn xuống).
<code>power</code>	Nâng các phần tử trong mảng thứ nhất lên lũy thừa của các phần tử tương ứng trong mảng thứ hai, theo từng phần tử.
<code>maximum</code> , <code>fmax</code>	Tối đa theo từng phần tử. <code>fmax</code> bỏ qua <code>NaN</code> .
<code>minimum</code> , <code>fmin</code>	Tối thiểu theo từng phần tử. <code>fmin</code> bỏ qua <code>NaN</code> .
<code>mod</code>	Phần dư theo từng phần tử (tương đương với toán tử <code>%</code> của Python).
<code>copysign</code>	Sao chép dấu của các giá trị trong mảng thứ hai vào các giá trị trong mảng thứ nhất, theo từng phần tử.
<code>greater</code> , <code>greater_equal</code> , <code>less</code> , <code>less_equal</code> , <code>equal</code> , <code>not_equal</code>	Thực hiện các phép so sánh theo từng phần tử, tạo ra một mảng boolean. Tương đương với các toán tử infix như <code>></code> , <code>>=</code> , <code>==</code> , v.v.
<code>logical_and</code> , <code>logical_or</code> , <code>logical_xor</code>	Thực hiện các phép toán logic AND, OR, và XOR theo từng phần tử. Tương đương với các toán tử infix <code>&</code> , <code> </code> , <code>^</code> .

5.4. Xử lý dữ liệu dựa trên mảng Numpy

Việc sử dụng các mảng NumPy (*NumPy arrays*) cho phép biểu diễn nhiều tác vụ xử lý dữ liệu dưới dạng các biểu thức mảng ngắn gọn và hiệu quả, vốn dĩ trong các ngôn ngữ lập trình thông thường thường phải triển khai thông qua các vòng lặp. Kỹ thuật chuyển đổi các thuật toán xử lý từng phần tử riêng lẻ sang các phép toán mảng toàn cục được gọi là **vector hóa** (*vectorization*).

Xét về hiệu năng, các phép toán đã được vector hóa trong NumPy thường cho tốc độ thực thi cao hơn đáng kể so với các phép toán tương đương được viết bằng Python thuần túy, do tận dụng được các tối ưu hóa ở mức thấp và loại bỏ chi phí lặp lại không cần thiết.

Ví dụ dưới đây minh họa cách sử dụng NumPy để đánh giá biểu thức hàm số:

$$[f(x, y) = \sqrt{x^2 + y^2}]$$

trên một lưới điểm đều nhau trong mặt phẳng. Hàm `np.meshgrid` nhận vào hai mảng một chiều và tạo ra hai mảng hai chiều biểu diễn toàn bộ các cặp giá trị $((x, y))$ có thể hình thành từ các phần tử trong hai mảng đầu vào đó:

```
points = np.arange(-5, 5, 0.01) # 1000 điểm cách đều nhau
xs, ys = np.meshgrid(points, points)
ys
```

```
array([[ -5.   ,  -5.   ,  -5.   , ...,  -5.   ,  -5.   ,  -5.   ],
       [ -4.99,  -4.99,  -4.99, ...,  -4.99,  -4.99,  -4.99],
       [ -4.98,  -4.98,  -4.98, ...,  -4.98,  -4.98,  -4.98],
       ...,
       [  4.97,   4.97,   4.97, ...,   4.97,   4.97,   4.97],
       [  4.98,   4.98,   4.98, ...,   4.98,   4.98,   4.98],
       [  4.99,   4.99,   4.99, ...,   4.99,   4.99,   4.99]],
      shape=(1000, 1000))
```

Bây giờ, việc đánh giá hàm chỉ đơn giản là vấn đề viết cùng một biểu thức như bạn sẽ viết với hai điểm:

```
z = np.sqrt(xs ** 2 + ys ** 2)
z
```

```
array([[7.07106781, 7.06400028, 7.05693985, ..., 7.04988652, 7.05693985,
        7.06400028],
       [7.06400028, 7.05692568, 7.04985815, ..., 7.04279774, 7.04985815,
        7.05692568],
       [7.05693985, 7.04985815, 7.04278354, ..., 7.03571603, 7.04278354,
        7.04985815],
       ...,
       [7.04988652, 7.04279774, 7.03571603, ..., 7.0286414 , 7.03571603,
        7.04279774],
       [7.05693985, 7.04985815, 7.04278354, ..., 7.03571603, 7.04278354,
        7.04985815],
       [7.06400028, 7.05692568, 7.04985815, ..., 7.04279774, 7.04985815,
        7.05692568]], shape=(1000, 1000))
```

Có thể thấy rằng mảng được hình thành có kích thước `1000 * 1000` tương ứng với từng p

5.4.1. Biểu thức điều kiện với mảng logical

Hàm `numpy.where` là một phiên bản vector hóa của biểu thức `x if condition else y`. Giả sử chúng ta có hai mảng các giá trị và một mảng kiểu logical:

```
xarr = np.array([1.1, 1.2, 1.3, 1.4, 1.5])
yarr = np.array([2.1, 2.2, 2.3, 2.4, 2.5])
cond_where = np.array([True, False, True, True, False])
```

Giả sử chúng ta muốn lấy một giá trị từ `xarr` bất cứ khi nào giá trị tương ứng trong `cond_where` là `True` và nếu không thì lấy giá trị từ `yarr`. Một phương pháp để thực hiện việc này là dùng vòng lặp như sau:

```
result_lc = [(x if c else y)
              for x, y, c in zip(xarr, yarr, cond_where)]
result_lc
```

Mặc dù kết quả không sai, nhưng sẽ có nhiều vấn đề. Thứ nhất, tốc độ thực hiện sẽ chậm đặc biệt là khi làm việc với các mảng lớn. Thứ hai, sẽ cần nhiều vòng lặp để thực hiện trên các mảng nhiều chiều. Một cách hiệu quả để thực hiện việc này là dùng `np.where`. Câu lệnh sẽ ngắn gọn như sau:

```
result_where = np.where(cond_where, xarr, yarr)
result_where
```

Các tham số thứ hai và thứ ba trong `np.where` không nhất thiết phải là mảng; một hoặc cả hai tham số có thể là các giá trị đơn lẻ.

Một công dụng điển hình của `where` trong phân tích dữ liệu là tạo ra một mảng mới các giá trị dựa trên một mảng khác. Giả sử bạn có một ma trận dữ liệu được tạo ngẫu nhiên và bạn muốn thay thế tất cả các giá trị dương bằng số 2 và tất cả các giá trị âm bằng số -2. Điều này rất dễ thực hiện với `np.where`:

```
arr_rand_where = np.random.standard_normal((4, 4))
arr_rand_where
```

```
np.where(arr_rand_where > 0, 2, -2)
```

Bạn có thể kết hợp mảng với các giá trị riêng lẻ khi sử dụng `np.where`. Ví dụ, chúng ta có thể đặt tất cả các giá trị dương trong `arr_rand_where` thành 2 như sau:

```
np.where(arr_rand_where > 0, 2, arr_rand_where) # đặt chỉ các giá trị dương thành 2
```

Các mảng được sử dụng làm tham số thứ hai và thứ ba của `where` không nhất thiết phải có cùng `shape`, nhưng chúng phải có thể *lan truyền* đến một `shape` chung. Chúng ta sẽ thảo luận chi tiết hơn về các quy tắc lan truyền trong các phần tiếp theo. Mặc dù `where` không bị giới hạn ở việc sử dụng trên các mảng kiểu logical, nhưng hàm này đặc biệt hữu ích cho việc đó.

5.4.2. Các phương thức thống kê toán học

NumPy cung cấp một tập hợp phong phú các hàm toán học để tính toán các thống kê tổng hợp trên toàn bộ mảng hoặc dọc theo các trục cụ thể. Các phép toán này thường được gọi là phép toán tổng hợp (reductions) và có thể được truy cập thông qua hai cách: (i) dưới dạng phương thức (method) của đối tượng mảng, hoặc (ii) thông qua các hàm cấp cao (top-level functions) trong gói NumPy.

Các hàm phổ biến bao gồm `sum` (tổng), `mean` (trung bình), `std` (độ lệch chuẩn), `min` (giá trị nhỏ nhất), và `max` (giá trị lớn nhất). Khi sử dụng hàm cấp cao của NumPy, mảng dữ liệu cần được truyền vào như một tham số đầu tiên. Ngược lại, khi gọi phương thức trực tiếp từ đối tượng mảng, dữ liệu gốc đã được định nghĩa sẵn nên không cần truyền lại.

Ví dụ dưới đây tạo một mảng hai chiều gồm các giá trị ngẫu nhiên từ phân phối chuẩn chuẩn hóa (phân phối Gauss với kỳ vọng bằng 0 và độ lệch chuẩn bằng 1), và thực hiện một số phép tính tổng hợp:

```
arr_stats = np.random.standard_normal((5, 4))  
arr_stats
```

```
arr_stats.mean()
```

```
np.mean(arr_stats)
```

```
arr_stats.sum()
```

Các hàm như `mean` và `sum` nhận một tham số `axis` tùy chọn để tính toán thống kê trên trục được chỉ định, dẫn đến một mảng có ít hơn một chiều:

```
arr_stats.mean(axis=1)
```

```
arr_stats.sum(axis=0)
```

Ở đây, `arr_stats.mean(axis=1)` có thể hiểu là "tính trung bình trên các cột" và `arr_stats.sum(axis=0)` có thể hiểu là "tính tổng trên các hàng". Các phương thức khác như `cumsum` và `cumprod` không tổng hợp, thay vào đó tạo ra một mảng các kết quả trung gian:

```
arr_cumsum = np.array([0, 1, 2, 3, 4, 5, 6, 7])  
arr_cumsum.cumsum()
```

Trong các mảng nhiều chiều, các hàm tích lũy như `cumsum` trả về một mảng có cùng kích thước nhưng với các tổng hoặc tích một phần được tính toán dọc theo trục được chỉ định theo các phần tử "thấp hơn":

```
arr_cumsum_2d = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
arr_cumsum_2d
```

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
arr_cumsum_2d.cumsum(axis=0)
```

```
array([[ 0,  1,  2],
       [ 3,  5,  7],
       [ 9, 12, 15]])
```

```
arr_cumsum_2d.cumsum(axis=1)
```

Bảng ... cung cấp một danh sách các phương thức tổng hợp và thống kê mảng có sẵn. Chúng ta sẽ xem xét nhiều phương thức này chi tiết hơn trong các chương sau.

Bảng ... các phương thức thống kê mảng cơ bản

Phương thức	Mô tả
<code>sum</code>	Tổng của tất cả các phần tử trong mảng hoặc dọc theo một trục. Các mảng có kích thước zero có tổng là 0.
<code>mean</code>	Trung bình số học. Các mảng có kích thước zero có <code>NaN</code> làm trung bình.
<code>std</code> , <code>var</code>	Độ lệch chuẩn và phương sai, tương ứng, với khả năng điều chỉnh bậc tự do của mẫu số.
<code>min</code> , <code>max</code>	Giá trị tối thiểu và tối đa.
<code>argmin</code> , <code>argmax</code>	Chỉ mục của các phần tử tối thiểu và tối đa, tương ứng.
<code>cumsum</code>	Tổng tích lũy của các phần tử bắt đầu từ 0.
<code>cumprod</code>	Tích tích lũy của các phần tử bắt đầu từ 1.

```
### Các phương thức cho mảng kiểu logical
<hr>
```

Các giá trị kiểu logical sẽ được biến đổi thành 1 (`True`) và 0 (`False`) trong các phương thức trước đó. Do đó, `sum` thường được sử dụng như một phương tiện để đếm các giá trị `True` trong một mảng kiểu logical:

```
arr_bool_sum = np.random.standard_normal(100)
(arr_bool_sum > 0).sum() # Số lượng các giá trị dương
```

```
(arr_bool_sum <= 0).sum() # Số lượng các giá trị không dương
```

Có hai phương thức khác, bao gồm `any` và `all`, đặc biệt hữu ích cho các mảng kiểu logical.

- `any` kiểm tra xem có ít nhất một giá trị `True` trong mảng hay không.
- `all` kiểm tra xem mọi giá trị có phải là `True` hay không:

```
bools = np.array([False, False, True, False])
bools.any()
```

```
bools.all()
```

Các phương thức này cũng hoạt động với các mảng không phải kiểu logical. Nguyên tắc là các giá trị sẽ được biến đổi về kiểu logical trước khi các phương thức được gọi. Bạn đọc tham khảo cách biến đổi các giá trị thành logical ở phần trên.

```
### Sắp xếp giá trị trong mảng Numpy
<hr>
```

Giống như kiểu danh sách tích hợp sẵn của Python, các mảng Numpy có thể được sắp xếp tại chỗ bằng phương thức `sort`:

```
arr_sort = np.random.standard_normal(6)
arr_sort
```

```
arr_sort.sort()
arr_sort
```

Bạn đọc cũng có thể sắp xếp mỗi thành phần của một mảng nhiều chiều dọc theo một trục bằng cách truyền số trục cho `sort`:

```
arr_sort_2d = np.random.standard_normal((5, 3))
arr_sort_2d
```

```
array([[ -0.96248719, -1.02599251, -0.24977692],
       [  0.61971614, -0.37657236,  0.98163527],
       [  0.51902348,  0.01875717,  0.13542396],
       [-0.1436972 ,  0.43719963,  1.27441518],
       [  0.23134265, -1.97811727, -0.52349906]])
```

```
arr_sort_2d_copy = arr_sort_2d.copy()
arr_sort_2d_copy.sort(axis=0) # Sắp xếp các cột
arr_sort_2d_copy
```

```
array([[ -0.96248719, -1.97811727, -0.52349906],
       [-0.1436972 , -1.02599251, -0.24977692],
       [  0.23134265, -0.37657236,  0.13542396],
       [  0.51902348,  0.01875717,  0.98163527],
       [  0.61971614,  0.43719963,  1.27441518]])
```

```
arr_sort_2d_copy_2 = arr_sort_2d.copy()
arr_sort_2d_copy_2.sort(axis=1) # Sắp xếp các hàng
arr_sort_2d_copy_2
```

```
array([[ -1.02599251, -0.96248719, -0.24977692],
       [-0.37657236,  0.61971614,  0.98163527],
       [ 0.01875717,  0.13542396,  0.51902348],
       [-0.1436972 ,  0.43719963,  1.27441518],
       [-1.97811727, -0.52349906,  0.23134265]])
```

Hàm `np.sort` trả về một bản sao đã sắp xếp của một mảng thay vì sửa đổi mảng tại chỗ.

Một cách nhanh chóng và hiệu quả để tính toán các phân vị của một mảng là sắp xếp nó và chọn giá trị tại một hạng cụ thể. Ví dụ:

```
large_arr_sort = np.random.standard_normal(1000)
large_arr_sort.sort()
large_arr_sort[int(0.05 * len(large_arr_sort))] # 5% quantile
```

Để biết thêm chi tiết về việc sử dụng các tính năng sắp xếp của NumPy, và các thuật toán nâng cao hơn như sắp xếp gián tiếp, sẽ được trình bày ở phần sau của sách. Một số hàm khác của NumPy liên quan đến sắp xếp, như `lexsort`, không được đề cập ở đây vì chúng ít được sử dụng trong phân tích dữ liệu.

5.4.3. Các phép toán cho tập hợp sử dụng trên mảng

NumPy có một số phép toán tập hợp cơ bản dành cho các mảng. Một hàm thường được sử dụng là `np.unique`, trả về các phần tử duy nhất **đã được sắp xếp** trong một mảng:

```
names_set = np.array(["Bob", "Joe", "Will", "Bob", "Will", "Joe", "Joe"])
np.unique(names_set)
```

```
array(['Bob', 'Joe', 'Will'], dtype='<U4')
```

```
ints_set = np.array([3, 3, 3, 2, 2, 1, 1, 4, 4])
np.unique(ints_set)
```

```
array([1, 2, 3, 4])
```

Bạn đọc có thể so sánh `np.unique` với các hàm tích hợp sẵn của Python:

```
sorted(set(names_set))
```

Một hàm khác, `np.in1d`, dùng để kiểm tra nếu các giá trị trong một mảng trong một mảng khác, và giá trị trả về là một mảng kiểu logical:

```
values_set = np.array([6, 0, 0, 3, 2, 5, 6])
np.in1d(values_set, [2, 3, 6])
```


Xem Bảng ... để biết danh sách các hàm tập hợp của NumPy cho các mảng một chiều (`1D`).

Bảng ... Các phép toán tập hợp mảng

Phương thức	Mô tả
<code>unique(x)</code>	Tính toán các phần tử duy nhất đã được sắp xếp trong <code>x</code> .
<code>intersect1d(x, y)</code>	Tính toán các phần tử chung đã được sắp xếp trong <code>x</code> và <code>y</code> .
<code>union1d(x, y)</code>	Tính toán hợp của các phần tử đã được sắp xếp.
<code>in1d(x, y)</code>	Tính toán một mảng boolean cho biết liệu mỗi phần tử của <code>x</code> có chứa trong <code>y</code> hay không.
<code>setdiff1d(x, y)</code>	Hiệu tập hợp; các phần tử trong <code>x</code> không có trong <code>y</code> .
<code>setxor1d(x, y)</code>	Hiệu đối xứng tập hợp; các phần tử nằm trong một trong các mảng nhưng không nằm trong cả hai.

5.5. Nhập và xuất file với mảng Numpy

Numpy có thể lưu và tải dữ liệu vào và từ đĩa ở định dạng văn bản hoặc nhị phân. Trong các phần sau, chúng ta sẽ thảo luận về các công cụ tích hợp sẵn của NumPy để đọc và ghi dữ liệu từ các mảng, nhưng hãy nhớ rằng pandas cung cấp nhiều hàm tuyệt vời để đọc các tệp văn bản hoặc dạng bảng vào các đối tượng `DataFrame`; những hàm đó sẽ được đề cập chi tiết hơn trong chương tiếp theo.

Các mảng được lưu vào đĩa ở **định dạng nhị phân** bằng `np.save` và được load bằng `np.load`:

```
arr_io = np.arange(10)
np.save("some_array", arr_io)
```

Nếu tên tệp chưa có phần mở rộng `.npy`, Python sẽ được tự động thêm vào khi lưu. Để tải dữ liệu mảng, chúng ta sử dụng `np.load`:

```
np.load("some_array.npy")
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Bạn đọc có thể lưu nhiều mảng trong một kho lưu trữ định dạng zip không nén bằng `np.savez` và truyền các mảng dưới dạng tham số như sau:

```
np.savez("array_archive.npz", a=arr_io, b=arr_io)
```

Khi tải một tệp `.npz`, bạn đọc nhận được một đối tượng giống như từ điển và có thể gọi các mảng riêng lẻ bằng tham số đã khai báo lúc lưu:

```
arch = np.load("array_archive.npz")  
arch["b"]
```

```
### Lưu và tải tệp văn bản  
<hr>
```

Việc tải dữ liệu văn bản từ các tệp có thể được thực hiện bằng các hàm như `np.loadtxt` hoặc hàm `np.genfromtxt` tổng quát hơn. Các hàm này có nhiều tùy chọn, nhưng cách sử dụng điển hình là tải các tệp văn bản được phân tách bằng dấu phân cách (như CSV). Hãy xem xét một tệp CSV như sau:

```
1,2,3,4,foo  
5,6,7,8,bar  
9,10,11,12,baz
```

Tệp này có thể được tải vào một mảng NumPy hai chiều như sau:

```
# Để chạy ví dụ này, bạn cần tạo một file tên là array_ex.txt
# với nội dung như sau:
# 1,2,3,4, # Bỏ qua foo vì loadtxt mặc định chỉ đọc số
# 5,6,7,8,
# 9,10,11,12

# with open("array_ex.txt", "w") as f:
#     f.write("1,2,3,4\n")
#     f.write("5,6,7,8\n")
#     f.write("9,10,11,12\n")

# arr_loadtxt = np.loadtxt("array_ex.txt", delimiter=",")
# arr_loadtxt

# np.savetxt("my_array_saved.csv", arr_loadtxt, delimiter=",")
```

`np.savetxt` thực hiện thao tác ngược lại: ghi một mảng vào một tệp văn bản được phân tách bằng dấu phân tách. Hàm `np.genfromtxt` tương tự như `np.loadtxt` nhưng hướng đến các mảng có cấu trúc và xử lý dữ liệu bị thiếu. Chúng ta sẽ không sử dụng các hàm này thường xuyên trong cuốn sách này, vì các công cụ của pandas để đọc các tệp văn bản, như `pandas.read_csv`, thường thuận tiện hơn. Đối với dữ liệu rất lớn, những hàm này có thể không đủ nhanh, vì vậy bạn có thể cần phải phát triển một trình phân tích tệp tùy chỉnh hoặc sử dụng một công cụ khác.

5.6. Các phép toán đại số tuyến tính

Các phép toán đại số tuyến tính, như nhân ma trận, phân tích, định thức và các hàm toán học ma trận vuông khác, là một phần quan trọng các thư viện dành riêng cho tính toán mảng. Không giống như một số ngôn ngữ như MATLAB, việc nhân hai mảng hai chiều bằng `*` là một phép nhân theo từng phần tử thay vì một phép nhân ma trận. Do đó, hàm `dot`, cả dưới dạng một hàm trong không gian tên `numpy` và một phương thức `ndarray`, để thực hiện nhân ma trận:

```
x_linalg = np.array([[1., 2., 3.], [4., 5., 6.]])
y_linalg = np.array([[6., 23.], [-1, 7], [8, 9]])
x_linalg
```

```
y_linalg
```

```
x_linalg.dot(y_linalg)
```

`x_linalg.dot(y_linalg)` tương đương với `np.dot(x_linalg, y_linalg)`:

```
np.dot(x_linalg, y_linalg)
```

Một phép nhân ma trận giữa một mảng hai chiều và một mảng một chiều có kích thước phù hợp sẽ dẫn đến một mảng một chiều:

```
np.dot(x_linalg, np.ones(3))
```

Toán tử `@` cũng có thể được sử dụng như một toán tử nhân ma trận:

```
x_linalg @ np.ones(3)
```

Thư viện con `numpy.linalg` có một tập hợp chuẩn các phép phân tích ma trận và những phép toán thường dùng như nghịch đảo và định thức.

```
from numpy.linalg import inv, qr
X_inv = np.random.standard_normal((5, 5))
mat_inv = X_inv.T @ X_inv # .T là một bí danh cho transpose
inv(mat_inv)
```

```
array([[13.8207132 ,  2.82454621,  5.18432395, -1.04799322,  3.56055482],
       [ 2.82454621,  1.04015628,  1.33922863, -0.13589287,  1.1953729 ],
       [ 5.18432395,  1.33922863,  2.25203873, -0.35134988,  1.76006171],
       [-1.04799322, -0.13589287, -0.35134988,  0.24614031, -0.13208114],
       [ 3.56055482,  1.1953729 ,  1.76006171, -0.13208114,  1.9306511 ]])
```

```
mat_inv @ inv(mat_inv)
```

Hàm `inv` tính toán nghịch đảo của một ma trận vuông. Phép nhân `mat_inv @ inv(mat_inv)` sẽ tạo ra ma trận đơn vị hoặc một ma trận rất gần với ma trận đơn vị nếu phép tính toán nghịch đảo là gần đúng.

Hàm `qr` tính toán phân rã của một ma trận:

```
q, r = qr(mat_inv)
r
```

```
array([[ -2.50044755, -3.83375318, 10.68482954,  0.21047701, -2.83390096],
       [ 0.          , -5.2601914 ,  2.61651598,  2.73324026,  1.20878842],
       [ 0.          ,  0.          , -3.25844996, -6.44064825,  2.79676436],
       [ 0.          ,  0.          ,  0.          , -2.13919745, -0.49012504],
       [ 0.          ,  0.          ,  0.          ,  0.          ,  0.21848261]])
```

Bảng ... liệt kê một số các hàm đại số tuyến tính thường được sử dụng. Bạn đọc có thể tham khảo tài liệu của `numpy.linalg` để biết thêm chi tiết.

Bảng ... các hàm `numpy.linalg` thường được sử dụng

Hàm	Mô tả
<code>diag</code>	Trả về các phần tử đường chéo (hoặc ngoài đường chéo) của một mảng vuông dưới dạng một mảng một chiều, hoặc chuyển đổi một mảng một chiều thành một ma trận vuông với các phần tử đó trên đường chéo, các phần tử khác bằng không.
<code>dot</code>	Nhân ma trận.
<code>trace</code>	Tính tổng các phần tử đường chéo.
<code>det</code>	Tính định thức ma trận.
<code>eig</code>	Tính các giá trị riêng (eigenvalues) và các vector riêng (eigenvectors) của một ma trận vuông.
<code>inv</code>	Tính nghịch đảo của một ma trận vuông.
<code>pinv</code>	Tính nghịch đảo giả Moore-Penrose của một ma trận.
<code>qr</code>	Tính phân rã QR.
<code>svd</code>	Tính phân rã giá trị suy biến (Singular Value Decomposition - SVD).
<code>solve</code>	Giải hệ phương trình tuyến tính $Ax = b$ cho x , trong đó A là một ma trận vuông.
<code>lstsq</code>	Tính nghiệm bình phương tối thiểu theo phương pháp bình phương tối thiểu cho $Ax = b$.

```
samples_rng = np.random.standard_normal(size=(4, 4))
samples_rng
```

Mô-đun `random` tích hợp sẵn của Python, ngược lại, chỉ lấy mẫu một giá trị tại một thời điểm. Như bạn có thể thấy từ phép đo benchmark này, `numpy.random` nhanh hơn nhiều để tạo ra số lượng lớn các số ngẫu nhiên:

```
from random import normalvariate
N = 1_000_000
# %timeit samples = [normalvariate(0, 1) for _ in range(N)]
# %timeit np.random.standard_normal(N)

# Chạy các lệnh trên trong Jupyter Notebook / IPython để thấy kết quả timeit
# Ví dụ:
t0 = time.time()
samples_py_rng = [normalvariate(0, 1) for _ in range(N)]
t1 = time.time()
print(f"Python random: {t1 - t0:.4f} s")

t0 = time.time()
samples_np_rng = np.random.standard_normal(N)
t1 = time.time()
print(f"NumPy random: {t1 - t0:.4f} s")
```

Chúng ta nói rằng những con số này là *giả ngẫu nhiên* (pseudorandom) vì chúng được tạo ra bởi một thuật toán với hành vi xác định dựa trên *mầm* (seed) của trình tạo số ngẫu nhiên. Bạn có thể thay đổi mầm sinh số ngẫu nhiên của NumPy bằng `numpy.random.seed` (cách cũ) hoặc sử dụng `numpy.random.default_rng` (cách mới được khuyến nghị):

```
rng = np.random.default_rng(seed=12345)
data_rng = rng.standard_normal((2,3))
data_rng
```

Trình tạo số ngẫu nhiên (RNG) trong `numpy.random` là một đối tượng lưu trữ trạng thái bên trong. Khi bạn gọi các hàm như `rng.standard_normal`, trình tạo này sẽ nâng cao trạng thái của nó. Bạn có thể lấy một phiên bản trình tạo riêng biệt với các phiên bản khác trong mã của mình:

```
rng2 = np.random.default_rng(seed=12345)
data2_rng = rng2.standard_normal((2,3))
data2_rng
```

Như bạn có thể thấy, `data_rng` và `data2_rng` là giống hệt nhau. Các hàm dữ liệu trong `numpy.random` (ví dụ, `standard_normal`, `uniform`, `randn`) sử dụng một trình tạo ngẫu nhiên toàn cục, được tạo ngầm. Để có khả năng tái tạo tốt hơn trên các phiên bản NumPy, tôi khuyên bạn nên sử dụng một thể hiện của `numpy.random.Generator`, được tạo bằng `numpy.random.default_rng`.

Xem Bảng 4.8 để biết danh sách một phần các hàm có sẵn trong `numpy.random`. Tôi sẽ quay lại các hàm này trong các ví dụ sau của cuốn sách.

Bảng ... một số hàm `numpy.random`

Hàm	Mô tả
<code>seed</code>	Gieo mầm cho trình tạo số ngẫu nhiên. Không được sử dụng với <code>default_rng</code> .
<code>permutation</code>	Trả về một hoán vị ngẫu nhiên của một chuỗi, hoặc trả về một phạm vi đã hoán vị.
<code>shuffle</code>	Hoán vị ngẫu nhiên một chuỗi tại chỗ.
<code>uniform</code>	Rút các mẫu từ một phân phối đều.
<code>integers</code>	Rút các số nguyên ngẫu nhiên từ một phạm vi cho trước.
<code>standard_normal</code>	Rút các mẫu từ một phân phối chuẩn với trung bình 0 và độ lệch chuẩn 1 (dạng ma trận).
<code>binomial</code>	Rút các mẫu từ một phân phối nhị thức.
<code>normal</code>	Rút các mẫu từ một phân phối chuẩn (Gaussian).
<code>beta</code>	Rút các mẫu từ một phân phối beta.
<code>chisquare</code>	Rút các mẫu từ một phân phối chi bình phương.
<code>gamma</code>	Rút các mẫu từ một phân phối gamma.
<code>poisson</code>	Rút các mẫu từ một phân phối Poisson.
<code>rand</code>	Rút các mẫu từ một phân phối đều [0, 1).
<code>randn</code>	Rút các mẫu từ một phân phối "chuẩn" bình thường (trung bình 0, độ lệch chuẩn 1).

Mặc dù nhiều tính năng của Numpy được sử dụng trong pandas và các thư viện khác, việc hiểu rõ hơn về các mảng NumPy và tính toán véc-tơ sẽ giúp bạn đọc sử dụng các công cụ như pandas hiệu quả hơn. Các `ndarray` cung cấp một đối tượng lưu trữ dữ liệu mạnh mẽ, linh hoạt và hiệu suất cao, có thể dễ dàng định hình lại và xử lý bằng nhiều loại thuật toán. Khả năng thực hiện các phép toán toán học phức tạp trên toàn bộ khối dữ liệu mà không cần viết vòng lặp `for` làm cho NumPy trở thành một thành phần trung tâm trong hệ sinh thái tính toán khoa học của Python.

Trong chương tiếp theo, chúng ta sẽ tìm hiểu sâu hơn về pandas, thư viện cho phép chúng ta sẽ áp dụng nhiều khái niệm này vào các tình huống phân tích dữ liệu thực tế hơn.